Bab 5 Registrasi, Autentikasi, Otorisasi, dan Operasi Data

Tujuan Pembelajaran

Setelah mempelajari bab ini, Anda mampu:

1. **(C2 – Memahami)** menjelaskan perbedaan registrasi, verifikasi email, autentikasi, otorisasi, validasi masukan, dan pencatatan audit.
2. **(C3 – Menerapkan)** menerapkan registrasi, verifikasi email, autentikasi berbasis sesi, proteksi CSRF, serta validasi dengan *Form Request* pada Laravel.
3. **(C3 – Menerapkan)** menerapkan operasi *create*, *read*, *update*, dan *delete* (CRUD) yang tervalidasi dan terisolasi berdasarkan tenant.
4. (C3 – Menerapkan) memindahkan logika bisnis dari *controller* ke kelas *service* yang dapat diuji tanpa HTTP dengan memanfaatkan *dependency injection* Laravel.
5. **(C4 – Menganalisis)** menganalisis pemetaan peran, izin, pemisahan tugas, dan risiko pemberian hak akses berlebihan.
6. **(C5 – Mengevaluasi)** mengevaluasi aturan bisnis bertingkat waktu dan *audit trail* untuk menjaga konsistensi data historis.
7. **(C6 – Mencipta)** merancang alur identitas pengguna, otorisasi, operasi data, dan pencatatan perubahan pada aplikasi multi-tenant.

Orientasi Bab

Pada Bab 4, Anda telah mempelajari cara menjaga agar data antar-tenant tidak saling terlihat. Namun, isolasi tenant saja belum cukup. Aplikasi juga perlu memastikan bahwa pengguna yang masuk benar-benar memiliki identitas yang sah, hanya dapat menjalankan tindakan sesuai perannya, serta meninggalkan jejak ketika mengubah data penting.

Bayangkan aplikasi kantin dengan beberapa jenis pengguna: pengelola kantin, operator tenant, kasir, dan staf dapur. Kasir perlu membuat pesanan, tetapi tidak seharusnya mengubah tarif komisi. Operator tenant dapat mengelola menu di tenantnya sendiri, tetapi tidak boleh mengakses data tenant lain. Pengelola kantin dapat mengatur tenant di kantinnya, tetapi tindakan istimewanya tetap perlu dibatasi dan dicatat.

Bab ini membahas alur lengkap dari pembuatan akun hingga operasi data: registrasi, verifikasi email, autentikasi, perlindungan CSRF, otorisasi berbasis peran, CRUD tervalidasi, pemindahan logika bisnis dari *controller* ke kelas *service*, penugasan peran, aturan bisnis berlaku efektif, dan *audit trail*.

> **Peta bab.** Contoh kode pada bab ini diambil dari proyek pendamping *Kantin Multi-Tenant* (tag `pertemuan-05`). Beberapa contoh generik disajikan lebih dahulu untuk menjelaskan konsep, lalu dibandingkan dengan implementasi nyata pada proyek. Seluruh angka dan studi kasus bersifat **ilustratif**.
>
> **Prasyarat.** Bab ini melanjutkan Bab 4. Seluruh contoh yang mengakses data milik tenant mengasumsikan aplikasi telah menerapkan `TenantContext`, *global scope* tenant, serta *policy* kepemilikan tenant secara benar.

5.1 Siklus Identitas, Akses, dan Data

Sebelum mempelajari kode, Anda perlu membedakan beberapa istilah yang sering digunakan secara bergantian.

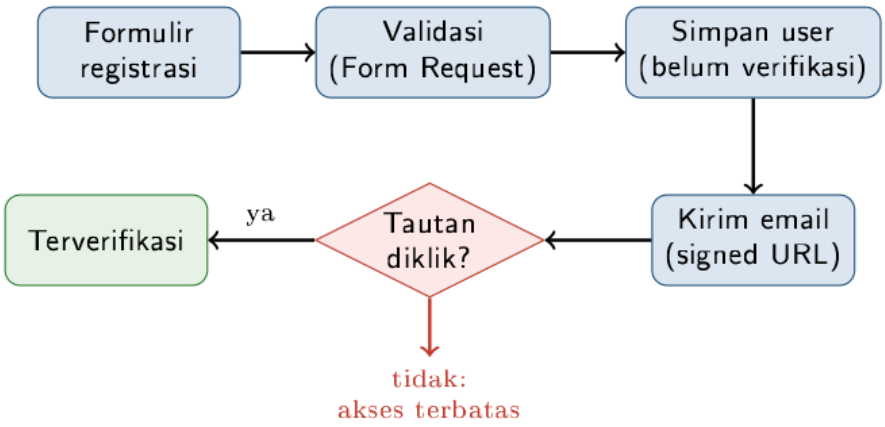
Konsep	Pertanyaan yang dijawab	Contoh
Registrasi	Siapa yang ingin memiliki akun?	Pengguna membuat akun dengan nama, email, dan kata sandi
Verifikasi email	Apakah pengguna menguasai alamat email tersebut?	Pengguna membuka tautan verifikasi dari email
Autentikasi	Apakah pengguna benar-benar pemilik akun itu?	Pengguna masuk dengan email dan kata sandi
Otorisasi	Apa yang boleh dilakukan pengguna setelah masuk?	Kasir boleh membuat pesanan, tetapi tidak mengubah tarif komisi
Validasi	Apakah data yang dikirim memenuhi aturan sistem?	Harga menu harus bilangan bulat tidak negatif
Audit trail	Siapa mengubah apa dan kapan?	Sistem mencatat perubahan tarif komisi oleh pengelola kantin

Tabel 5.1. Perbedaan konsep dalam siklus identitas dan pengelolaan data.

Urutan pada Tabel 5.1 penting dan tidak dapat dibalik. Aplikasi tidak seharusnya memeriksa izin sebelum mengetahui siapa pengguna yang masuk. Demikian pula, aplikasi tidak boleh menyimpan data sebelum memeriksa validitas masukan dan hak akses pengguna.

5.2 Registrasi dan Verifikasi Akun

Registrasi menciptakan akun baru. Verifikasi email menegaskan bahwa pemilik akun dapat mengakses alamat email yang digunakan saat registrasi. Keduanya membentuk dasar pengelolaan identitas pengguna.



Gambar 5.1. Alur registrasi hingga verifikasi email.

Sumber: diadaptasi penulis, ilustratif.

5.2.1 Merancang Registrasi yang Minimal dan Tervalidasi

Formulir registrasi sebaiknya hanya meminta data yang benar-benar dibutuhkan pada tahap awal. Pengumpulan atribut yang terlalu banyak meningkatkan kesalahan pengisian dan beban pengguna.

Atribut	Aturan validasi minimum	Tujuan
`name`	Wajib, teks, maksimal 255 karakter	Menyimpan nama pengguna
`email`	Wajib, format email, unik pada tabel `users`	Mencegah akun dengan alamat email ganda
`password`	Wajib, terkonfirmasi, memenuhi aturan kata sandi bawaan	Melindungi akun dari kata sandi lemah
`email_verified_at`	Bernilai `null` saat akun dibuat	Menandai bahwa email belum diverifikasi

Tabel 5.2. Aturan minimum registrasi pengguna pada proyek pendamping.

Kata sandi tidak boleh disimpan sebagai teks polos. Sistem harus menyimpannya sebagai *hash* satu arah, sehingga sistem dapat memverifikasi kata sandi tanpa menyimpan bentuk aslinya.

Proyek pendamping tidak menulis *controller* registrasi sendiri, melainkan memakai paket **Laravel Fortify** yang menyediakan rute, pengendali, dan alur autentikasi bawaan. Yang perlu Anda tulis hanyalah aturan validasi dan cara membuat pengguna. Berkasnya adalah `app/Actions/Fortify/CreateNewUser.php`.

Kode 5.1 — app/Actions/Fortify/CreateNewUser.php

```php
<?php

namespace App\Actions\Fortify;

use App\Concerns\PasswordValidationRules;
use App\Concerns\ProfileValidationRules;
use App\Models\User;
use Illuminate\Support\Facades\Validator;
use Laravel\Fortify\Contracts\CreatesNewUsers;

class CreateNewUser implements CreatesNewUsers
{
    use PasswordValidationRules, ProfileValidationRules;

    /** @param array<string, string> $input */
    public function create(array $input): User
    {
        Validator::make($input, [
            ...$this->profileRules(),
            'password' => $this->passwordRules(),
        ])->validate();

        return User::create([
            'name' => $input['name'],
            'email' => $input['email'],
            'password' => $input['password'],
        ]);
    }
}
```

Aturan validasinya sendiri dipisahkan ke dua *trait* agar dapat dipakai ulang oleh halaman profil, bukan hanya oleh registrasi. Berkasnya adalah `app/Concerns/PasswordValidationRules.php` dan `app/Concerns/ProfileValidationRules.php`.

Kode 5.2 — Aturan validasi yang dapat dipakai ulang

```php
// app/Concerns/PasswordValidationRules.php
protected function passwordRules(): array
{
    return ['required', 'string', Password::default(), 'confirmed'];
}

// app/Concerns/ProfileValidationRules.php
protected function emailRules(?int $userId = null): array
{
    return [
        'required', 'string', 'email', 'max:255',
        $userId === null
            ? Rule::unique(User::class)
            : Rule::unique(User::class)->ignore($userId),
    ];
}
```

Perhatikan dua keputusan pada Kode 5.2. Pertama, `Password::default()` memusatkan aturan kekuatan kata sandi pada satu tempat sehingga dapat diperketat sekaligus untuk seluruh aplikasi. Kedua, parameter `$userId` pada `emailRules()` membuat aturan unik dapat mengabaikan pengguna yang sedang menyunting profilnya sendiri — tanpa itu, pengguna tidak dapat menyimpan profil tanpa mengganti email.

Model `User` memastikan kata sandi tersimpan sebagai *hash* melalui konversi tipe. Berkasnya adalah `app/Models/User.php`.

Kode 5.3 — app/Models/User.php (potongan konfigurasi dasar)

```php
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
    ];
}
```

> **Peringatan.** Jangan memasukkan atribut administratif sensitif seperti `role`, `status`, atau `tenant_id` ke dalam data registrasi publik tanpa aturan bisnis dan otorisasi yang jelas. Atribut tersebut tidak boleh dikendalikan langsung oleh pengguna baru. Pada Kode 5.1, perhatikan bahwa `User::create()` hanya menerima tiga atribut yang dipilih secara eksplisit, bukan seluruh isi `$input`.

5.2.2 Verifikasi Email

Verifikasi email membuktikan bahwa pengguna memiliki akses terhadap alamat email yang didaftarkan. Model pengguna mengimplementasikan `MustVerifyEmail`, lalu Laravel menyediakan rute dan notifikasi verifikasi.

Pada proyek pendamping, fitur ini diaktifkan melalui daftar fitur Fortify pada berkas `config/fortify.php`.

Kode 5.4 — config/fortify.php (potongan daftar fitur)

```php
'features' => [
    Features::registration(),
```

```
    Features::resetPasswords(),
    Features::emailVerification(),
    Features::twoFactorAuthentication([
        'confirm' => true,
        'confirmPassword' => true,
    ]),
    Features::passkeys([
        'confirmPassword' => true,
    ]),
],
```

Rute yang hanya boleh diakses pengguna terverifikasi dilindungi dengan middleware `verified`. Pada proyek pendamping, grup rute internal — tenant dan admin — memuat middleware ini melalui kelas `app/Support/Routing/PortalRoutes.php` yang dipanggil dari `bootstrap/app.php`, seperti yang Anda lihat pada Bab 4. Grup pelanggan sengaja tidak memakainya karena pelanggan kantin tidak memiliki akun.

Kode 5.5 — routes/web.php (potongan rute terlindungi)

```
Route::middleware(['auth', 'verified'])->group(function () {
    Route::get('dashboard', DashboardRedirectController::class)->name('dashboard');
});
```

Rute `dashboard` diarahkan ke `DashboardRedirectController`, sebuah *controller* satu aksi yang mengalihkan pengguna sesuai perannya: pengelola kantin ke portal admin, operator ke dasbor tenant aktif miliknya, dan pengguna lain ke dasbor umum.

Apabila pengguna belum memverifikasi emailnya, Laravel mengarahkannya ke halaman pemberitahuan verifikasi. Setelah tautan yang sah dibuka, kolom `email_verified_at` terisi dan pengguna memperoleh akses ke rute tersebut.

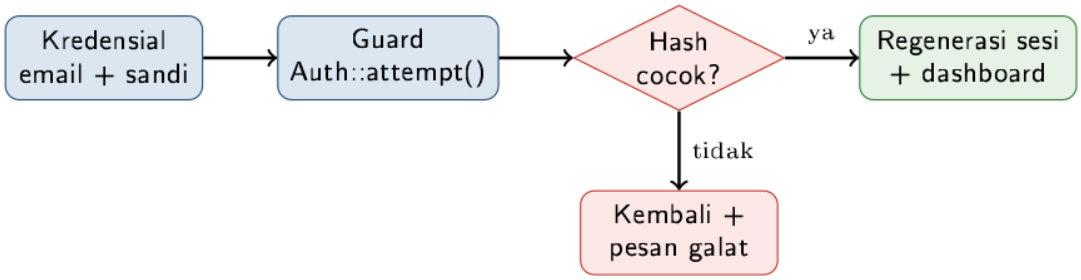
> **Tip.** Sediakan fitur pengiriman ulang email verifikasi beserta pembatasan frekuensinya. Pengguna dapat salah mengetik alamat email atau tidak segera menerima pesan karena penundaan layanan email.

5.3 Autentikasi, Sesi, dan Proteksi CSRF

Setelah memiliki akun, pengguna perlu membuktikan identitasnya melalui autentikasi. Pada aplikasi web berbasis sesi, autentikasi membangun sesi pengguna di peramban.

5.3.1 Proses Login

Autentikasi membandingkan kredensial yang diberikan pengguna dengan data yang tersimpan. Laravel menyediakan `Auth::attempt()` untuk memeriksa email dan kata sandi melalui *guard* yang dikonfigurasi.



Gambar 5.2. Alur autentikasi berbasis sesi dari kredensial hingga regenerasi sesi.

Sumber: diadaptasi penulis, ilustratif.

Contoh berikut menunjukkan bentuk *controller* login yang ditulis sendiri. Contoh ini disajikan untuk menjelaskan mekanismenya; proyek pendamping memakai Fortify sehingga langkah-langkah tersebut sudah tersedia secara bawaan.

Kode 5.6 — Contoh generik: login dengan regenerasi sesi

```php
public function store(LoginRequest $request)
{
    $credentials = $request->only('email', 'password');

    if (! Auth::attempt($credentials, $request->boolean('remember'))) {
        return back()
            ->withErrors(['email' => 'Email atau kata sandi tidak sesuai.'])
            ->onlyInput('email');
    }

    $request->session()->regenerate();

    return redirect()->intended(route('dashboard'));
}
```

Apabila autentikasi berhasil, aplikasi meregenerasi ID sesi. Langkah ini mengurangi risiko *session fixation*, yaitu kondisi ketika penyerang memaksa atau menebak ID sesi yang akan dipakai korban setelah login. Metode `intended()` mengarahkan pengguna ke halaman yang sebelumnya ingin diakses; jika tidak ada, pengguna diarahkan ke dasbor.

Perhatikan juga pesan galat pada contoh tersebut. Pesan yang sama dipakai untuk email yang tidak terdaftar maupun kata sandi yang salah, sehingga penyerang tidak dapat memakai halaman login untuk memetakan alamat email mana yang sudah terdaftar. Proyek pendamping menerapkan prinsip yang sama: pesan `Surel atau kata sandi tidak sesuai.` dipakai untuk surel yang tidak terdaftar, kata sandi yang salah, maupun akun yang dinonaktifkan (Kode 5.7).

> **Catatan.** Proyek pendamping menambahkan dua lapis lagi di atas kata sandi: autentikasi dua faktor dan *passkey*, keduanya diaktifkan pada Kode 5.4. Keduanya berada di luar cakupan bab ini, tetapi menunjukkan bahwa kata sandi bukan satu-satunya faktor identitas yang tersedia.

5.3.2 Pembatasan Laju Percobaan Login

Autentikasi yang benar pun masih dapat diserang melalui percobaan berulang. Proyek pendamping memasang dua lapis pertahanan pada berkas `app/Providers/FortifyServiceProvider.php`: pembatas laju per alamat IP dan penguncian per akun.

Kode 5.7 — app/Providers/FortifyServiceProvider.php (potongan autentikasi dan pembatasan laju)

```php
// configureAuthentication(): verifikasi kredensial + penguncian per akun
Fortify::authenticateUsing(function (Request $request): User {
    $lockout = app(LoginLockout::class);
    $email = (string) $request->input(Fortify::username());

    if ($lockout->isLocked($email)) {
        throw ValidationException::withMessages([
            Fortify::username() => 'Akun terkunci sementara karena terlalu banyak percobaan gagal. '
                .'Coba lagi dalam '.$lockout->minutesRemaining($email).' menit.',
        ]);
    }
```

```
    $user = User::query()->where('email', $email)->first();

    if ($user !== null && $user->isActive()
        && Hash::check((string) $request->input('password'), $user->password)) {
        $lockout->clear($email);

        return $user;
    }

    $lockout->recordFailure($email);

    throw ValidationException::withMessages([
        Fortify::username() => 'Surel atau kata sandi tidak sesuai.',
    ]);
});

// configureRateLimiting(): lapis per alamat IP
RateLimiter::for('login', function (Request $request) {
    return Limit::perMinute(20)->by($request->ip() ?? 'unknown');
});
```

Lapis pertama, pembatas laju bernama `login`, menerima paling banyak 20 percobaan per menit dari satu alamat IP untuk meredam serangan massal. Lapis kedua bekerja per akun: lima kegagalan dalam sepuluh menit mengunci akun selama lima belas menit, bahkan bila percobaan berikutnya memakai kata sandi yang benar. Akun nonaktif ditolak dengan pesan yang sama dengan kata sandi salah sehingga keberadaan akun tidak terbocorkan.

Perhatikan bahwa logika penguncian tidak ditulis di dalam *closure*, melainkan di kelas tersendiri, `app/Support/Auth/LoginLockout.php`. Pemisahan tanggung jawab seperti ini dibahas lebih lengkap pada Subbab 5.6.

Kode 5.8 — app/Support/Auth/LoginLockout.php (potongan)

```
final class LoginLockout
{
    public const MAX_FAILURES = 5;
    public const FAILURE_WINDOW_SECONDS = 600;
    public const LOCK_SECONDS = 900;

    public function recordFailure(string $email): bool
    {
        $now = now()->getTimestamp();
        $window = Cache::get($this->counterKey($email));

        if ($window === null || $now - $window['started_at'] >= self::FAILURE_WINDOW_SECONDS)
{
            $window = ['count' => 0, 'started_at' => $now];
        }

        $window['count']++;

        if ($window['count'] < self::MAX_FAILURES) {
            Cache::put($this->counterKey($email), $window, self::FAILURE_WINDOW_SECONDS);

            return false;
        }

        Cache::put($this->lockKey($email), $now + self::LOCK_SECONDS, self::LOCK_SECONDS);
        Cache::forget($this->counterKey($email));

        return true;
    }
```

```
    private function fingerprint(string $email): string
    {
        return hash('sha256', Str::lower(trim($email)));
    }
}
```

Kunci *cache* dibentuk dari *hash* SHA-256 alamat surel yang sudah dinormalisasi. Karena penghitungnya melekat pada akun, penyerang yang berganti alamat IP tidak dapat mengatur ulang hitungannya. Sebaliknya, beberapa pengguna sah yang berbagi satu jaringan — misalnya para kasir di satu kantin — tidak saling mengunci karena lapis per-IP diberi batas yang jauh lebih longgar.

> **Catatan.** Penguncian per akun memiliki sisi lain: penyerang dapat sengaja mengunci akun orang lain dengan memasukkan kata sandi salah berulang kali. Masa kunci yang singkat dan pesan yang menyebutkan sisa waktunya merupakan kompromi antara keamanan dan ketersediaan layanan.

5.3.3 Logout yang Aman

Siklus sesi belum lengkap tanpa *logout*. Proses ini perlu menghapus autentikasi pengguna, membatalkan sesi lama, dan menerbitkan token CSRF baru.

Kode 5.9 — Contoh generik: logout dan invalidasi sesi

```
public function destroy(Request $request)
{
    Auth::logout();

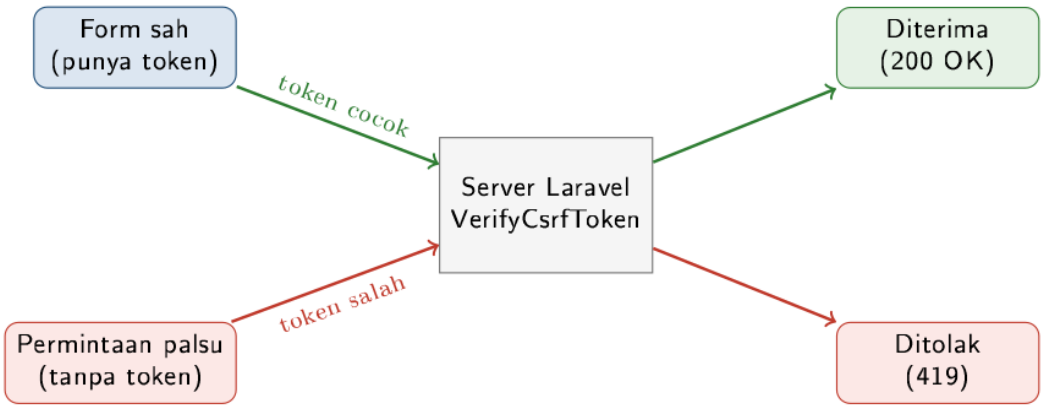
    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect()->route('login');
}
```

Ketiga langkah tersebut memiliki tugas yang berbeda. `Auth::logout()` melepaskan pengguna dari *guard*; `invalidate()` membuang seluruh isi sesi lama sehingga data yang tersimpan di dalamnya tidak dapat dipakai kembali; `regenerateToken()` menerbitkan token CSRF baru untuk sesi berikutnya. Invalidasi sesi penting terutama pada perangkat bersama, misalnya komputer kasir yang dipakai bergantian oleh beberapa staf.

5.3.4 Proteksi CSRF

Cross-Site Request Forgery (CSRF) adalah serangan yang berupaya membuat peramban pengguna yang telah masuk mengirim permintaan yang tidak dikehendaki ke aplikasi. Karena peramban ikut menyertakan *cookie* sesi secara otomatis, permintaan palsu itu tampak sah bagi server.



Gambar 5.3. Pemeriksaan token CSRF pada permintaan yang sah dan permintaan palsu.

Sumber: diadaptasi penulis, ilustratif.

Pada aplikasi berbasis sesi, Laravel memeriksa token CSRF untuk permintaan web yang mengubah data. Formulir Blade menambahkan token tersebut melalui direktif `@csrf`.

Kode 5.10 — Formulir menu dengan token CSRF

```
<form method="POST" action="{{ route('tenant.menus.store', $tenant) }}">
    @csrf

    <label for="name">Nama menu</label>
    <input id="name" type="text" name="name" value="{{ old('name') }}">

    @error('name')
        <span class="text-red">{{ $message }}</span>
    @enderror

    <button type="submit">Simpan</button>
</form>
```

Jika token dalam permintaan tidak cocok dengan token sesi, Laravel menolak permintaan tersebut. Pada konfigurasi standar aplikasi web Laravel, kegagalan ini muncul sebagai respons 419.

> **Catatan.** CSRF relevan terutama pada autentikasi berbasis *cookie* dan sesi peramban. API yang memakai token *bearer* dan tidak bergantung pada *cookie* sesi menggunakan pola perlindungan yang berbeda.

5.4 Otorisasi Berbasis Peran

Autentikasi menjawab pertanyaan “siapa pengguna ini?”. Otorisasi menjawab pertanyaan “apa yang boleh dilakukan pengguna ini?”. Pada aplikasi dengan banyak jenis pengguna, *role-based access control* (RBAC) membantu mengelola izin secara lebih terstruktur.

5.4.1 Konsep RBAC dan Pemisahan Tugas

RBAC menempatkan peran di antara pengguna dan izin. Seorang pengguna dapat memiliki satu atau beberapa peran, dan setiap peran memuat izin yang relevan dengan tugasnya.



Gambar 5.4. Struktur RBAC: pengguna, peran, dan izin dihubungkan oleh dua relasi banyak-ke-banyak.

Sumber: diadaptasi penulis, ilustratif.

Peran	Izin utama	Batas penting
Pengelola kantin	Administrasi kantin dan tenant di dalamnya	Terbatas pada kantin yang dikelolanya; tindakannya diaudit
Operator tenant	Mengelola menu, staf, dan laporan tenant	Tidak boleh mengakses tenant lain
Kasir	Membuat pesanan dan mengonfirmasi pembayaran	Tidak mengelola peran, menu, atau tarif komisi
Staf dapur	Melihat pesanan dan memperbarui statusnya	Tidak mengonfirmasi pembayaran atau mengelola staf

Tabel 5.3. Contoh pemetaan peran dan batas akses pada aplikasi Kantin.

Pemetaan ini menerapkan prinsip *least privilege*: pengguna memperoleh izin minimum yang diperlukan untuk menjalankan tugasnya. Pemetaan juga mendukung *separation of duty* atau pemisahan tugas — pihak yang mengubah tarif komisi sebaiknya bukan satu-satunya pihak yang menyetujui dan mengaudit perubahan tersebut.

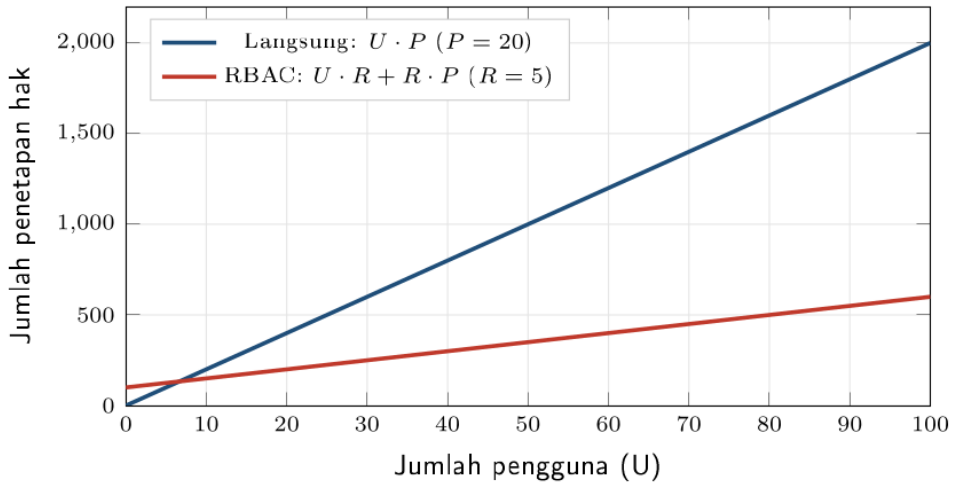
5.4.2 Model Ilustratif Jumlah Penetapan Akses

Untuk memahami mengapa RBAC menyederhanakan administrasi, kita dapat membandingkan jumlah penetapan hak pada dua pendekatan.

$$A_{\text{langsung}} = U \cdot P \qquad A_{\text{RBAC}} = U \cdot R + R \cdot P \tag{Persamaan 5.1}$$

dengan A(langsung) adalah jumlah penetapan bila izin diberikan langsung kepada setiap pengguna, A(RBAC) adalah jumlah penetapan pada model berbasis peran, U adalah jumlah pengguna, P adalah jumlah izin, dan R adalah jumlah peran.

Contoh 5.1. Untuk $U = 50$, $P = 20$, dan $R = 5$ diperoleh A(langsung) = 50 × 20 = 1.000 penetapan, sedangkan A(RBAC) = 50 × 5 + 5 × 20 = 250 + 100 = 350 penetapan. Penghematan ilustratifnya adalah (1.000 − 350) / 1.000 = 65%.



Gambar 5.5. Pertumbuhan jumlah penetapan hak terhadap jumlah pengguna pada dua pendekatan.

Sumber: diadaptasi penulis, data ilustratif.

Peringatan. Persamaan 5.1 mengasumsikan setiap pengguna memerlukan seluruh izin dan setiap peran memuat seluruh izin. Asumsi itu jarang berlaku pada sistem nyata, sehingga angka penghematannya bersifat ilustratif, bukan proyeksi.

Catatan. RBAC bukan alasan untuk membuat peran sebanyak mungkin. Terlalu banyak peran menimbulkan *role explosion*, yaitu kondisi ketika administrasi akses justru menjadi lebih sulit daripada sebelum RBAC diterapkan. Perhatikan pada Gambar 5.5 bahwa ketika jumlah pengguna sangat kecil, pendekatan berbasis peran bahkan dapat menghasilkan penetapan yang lebih banyak.

5.4.3 Peran, Gate, dan Policy pada Proyek Pendamping

Proyek pendamping tidak memakai tabel `roles` dan `permissions` yang lengkap. Peran disimpan pada dua tingkat yang berbeda, sesuai dengan dua konteks yang telah Anda kenal dari Bab 4.

Tingkat	Tempat penyimpanan	Dipakai untuk
Konteks aplikasi	Kolom `role` pada tabel `users`	Membedakan `admin` dan `tenant` pada grup rute
Keanggotaan kantin	Tabel `user_canteen_roles` (`owner`, `manager`, `staff`)	Keputusan administrasi pada `TenantPolicy`
Keanggotaan tenant	Tabel `user_tenant_roles` (`owner`, `operator`, `cashier`)	Sumber `TenantContext` dan otorisasi di dalam tenant

Tabel 5.4. Tiga tingkat peran pada proyek pendamping.

Pemeriksaan tingkat pertama dilakukan oleh *middleware* pada berkas `app/Http/Middleware/EnsureUserHasRole.php`.

Kode 5.11 — app/Http/Middleware/EnsureUserHasRole.php

```php
final class EnsureUserHasRole
{
    public function handle(Request $request, Closure $next, string $role): Response
    {
        $user = $request->user();

        // Guest ditangani middleware auth; di sini fokus pada role/status.
        if ($user === null || ! $user->hasRole($role)) {
            abort(403);
        }

        return $next($request);
    }
}

// app/Models/User.php
public function hasRole(string $role): bool
{
    return $this->role === $role && $this->isActive();
}
```

Perhatikan bahwa `hasRole()` memeriksa dua hal sekaligus: peran cocok **dan** akun berstatus aktif. Dengan begitu, menonaktifkan sebuah akun otomatis mencabut seluruh perannya tanpa perlu mengubah kolom `role`.

Keputusan yang lebih rinci dipusatkan pada *policy*, bukan disebar sebagai perbandingan string peran di banyak tempat. Berkasnya adalah `app/Policies/TenantPolicy.php`.

Kode 5.12 — app/Policies/TenantPolicy.php (potongan)

```php
class TenantPolicy
{
    private function managesCanteen(User $user, int $canteenId): bool
    {
        return UserCanteenRole::query()
            ->where('user_id', $user->id)
            ->where('canteen_id', $canteenId)
            ->whereIn('role', ['owner', 'manager', 'finance'])
            ->exists();
    }

    public function create(User $user): bool
    {
        return UserCanteenRole::query()
            ->where('user_id', $user->id)
            ->whereIn('role', ['owner', 'manager'])
            ->exists();
    }

    public function update(User $user, Tenant $tenant): bool
    {
        return $this->managesCanteen($user, $tenant->canteen_id);
    }

    public function scheduleCommission(User $user, Tenant $tenant): bool
    {
        return $this->managesCanteen($user, $tenant->canteen_id);
    }

    public function manageBank(User $user, Tenant $tenant): bool
    {
        return $this->managesCanteen($user, $tenant->canteen_id);
    }

    public function assignRole(User $user, Tenant $tenant): bool
    {
        return $this->managesCanteen($user, $tenant->canteen_id);
    }
}
```

Metode `create()` memakai daftar peran yang lebih sempit daripada metode lainnya. Membuat tenant baru hanya boleh dilakukan owner atau manager, sedangkan `update()`, `scheduleCommission()`, `manageBank()`, dan `assignRole()` memakai pemeriksaan `managesCanteen()` yang juga terbuka bagi peran finance. Perbedaan kecil seperti ini justru inti dari *least privilege*, dan jauh lebih mudah ditinjau ketika terkumpul di satu berkas.

Catatan. Laravel menyediakan dua mekanisme otorisasi yang saling melengkapi. *Gate* cocok untuk kemampuan umum yang tidak terikat objek tertentu, misalnya “boleh membuat menu”. *Policy* cocok untuk tindakan terhadap objek tertentu, karena ia menerima objeknya dan dapat memeriksa atribut seperti `tenant_id`.

Peringatan. Jangan memberikan *bypass* tanpa batas kepada peran istimewa melalui `Gate::before()` tanpa mendefinisikan ruang lingkup dan auditnya. Tindakan seperti perubahan peran, perubahan tarif, penghapusan permanen, atau ekspor lintas tenant justru memerlukan kontrol tambahan, bukan pengecualian.

5.4.4 Menegakkan Otorisasi pada Rute dan Tampilan

Penegakan akses harus berlangsung di sisi *server*. Menyembunyikan tombol pada antarmuka hanya memperbaiki pengalaman pengguna; tindakan itu tidak menghentikan pengguna yang mengirim permintaan HTTP secara langsung.

Pada proyek pendamping, otorisasi ditegakkan berlapis: middleware pada grup rute yang didefinisikan tunggal di kelas `PortalRoutes`, lalu policy pada setiap aksi — melalui `authorize()` di dalam Form Request atau `$this->authorize()` di awal metode *controller*.

Kode 5.13 — Penegakan otorisasi berlapis

```
// app/Support/Routing/PortalRoutes.php — lapis grup rute
public static function admin(Closure|string $routes): void
{
    Route::middleware(['web', 'auth', 'verified', 'role:admin'])
        ->prefix('admin')
        ->name('admin.')
        ->group($routes);
}

// app/Modules/Admin/Http/Requests/ScheduleCommissionRequest.php — lapis aksi (Form Request)
public function authorize(): bool
{
    return $this->user()?->can('scheduleCommission', $this->route('tenant')) ?? false;
}

// app/Modules/Admin/Http/Controllers/AdminTenantStatusController.php — lapis aksi
(controller)
$this->authorize('update', $tenant);
```

Pada tampilan Blade, direktif `@can` dipakai untuk menyembunyikan tautan yang tidak relevan bagi pengguna. Ingat bahwa direktif ini adalah penyempurna tampilan, bukan pengganti kedua lapis di atas.

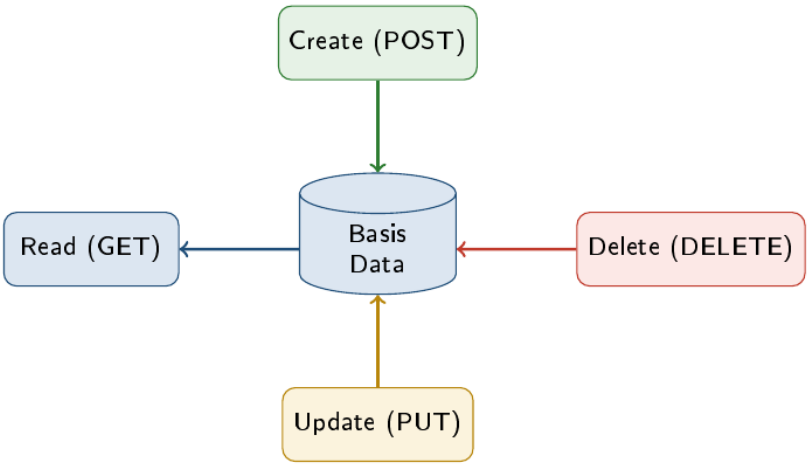
Kode 5.14 — Contoh: direktif otorisasi pada Blade

```
@can('create', App\Models\Tenant::class)
    <a href="{{ route('admin.tenants.create') }}">Tambah Tenant</a>
@endcan

@can('update', $tenant)
    <a href="{{ route('admin.tenants.edit', $tenant) }}">Ubah</a>
@endcan
```

5.5 CRUD yang Tervalidasi dan Terisolasi

CRUD adalah singkatan dari *create*, *read*, *update*, dan *delete*. Operasi ini menjadi inti pengelolaan menu, kategori, pesanan, staf, dan entitas aplikasi lain.



Gambar 5.6. Empat operasi CRUD dan tempat Form Request memvalidasi masukan.

Sumber: diadaptasi penulis, ilustratif.

Pada aplikasi multi-tenant, setiap operasi CRUD harus memenuhi tiga syarat sekaligus:

1. pengguna terautentikasi;
2. pengguna berwenang melakukan tindakan tersebut;
3. data berada dalam tenant yang benar.

Operasi	Verba HTTP	Contoh metode Eloquent	Contoh nama rute
Create	POST	`create()` atau `save()`	`tenant.menus.store`
Read	GET	`get()`, `find()`, `paginate()`	`tenant.menus.index`, `tenant.menus.show`
Update	PUT atau PATCH	`update()`	`tenant.menus.update`
Delete	DELETE	`delete()`	`tenant.menus.destroy`

Tabel 5.5. Pemetaan operasi CRUD pada Laravel.

5.5.1 Create: Menambah Data Secara Aman

Validasi dan otorisasi ditempatkan pada *Form Request*. Namun, validasi relasi harus tetap memperhatikan tenant aktif — inilah kesalahan paling sering pada aplikasi multi-tenant.

Kode 5.15 — Contoh: validasi pembuatan menu yang sadar tenant

```
class StoreMenuRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()?->can('create', Menu::class) ?? false;
    }

    public function rules(): array
    {
        $tenantId = app(TenantContext::class)->id();

        return [
            'name' => ['required', 'string', 'max:120'],
            'category_id' => [
```

```
                'required',
                Rule::exists('menu_categories', 'id')
                    ->where('tenant_id', $tenantId),
            ],
            'price_amount' => ['required', 'integer', 'min:0'],
            'stock_qty' => ['required', 'integer', 'min:0'],
        ];
    }
}
```

Aturan `category_id` pada Kode 5.15 tidak hanya memeriksa apakah kategori itu ada. Aturan tersebut juga memastikan kategori berada pada tenant aktif. Tanpa batasan `->where('tenant_id', $tenantId)`, pengguna dapat mengirim `category_id` milik tenant lain melalui formulir yang dimodifikasi, dan menu baru akan terhubung ke kategori yang bukan miliknya.

Nilai `tenant_id` tidak berasal dari formulir. Sesuai pola Bab 4, kolom itu diisi oleh *trait* `BelongsToTenant` dari konteks tenant aktif.

Peringatan. Jangan menambahkan `tenant_id` ke dalam `$fillable` apabila nilainya harus selalu ditentukan oleh konteks aplikasi. Pada proyek pendamping, aturan ini dinyatakan eksplisit dalam dokumentasi *trait*: `tenant_id` tidak boleh *mass-assignable*.

Catatan. Pada tingkat basis data, proyek pendamping menambahkan lapis terakhir berupa *composite foreign key* `(tenant_id, category_id)` yang merujuk `(tenant_id, id)` pada tabel `menu_categories`. Dengan begitu, meskipun validasi aplikasi terlewat, basis data tetap menolak menu yang terhubung ke kategori milik tenant lain.

5.5.2 Read: Membaca dan Memfilter Data

Pembacaan data perlu mendukung pencarian, pengurutan, dan penghalamanan. Semua kueri tetap berada dalam *scope* tenant aktif karena *global scope* bekerja otomatis.

Kode 5.16 — Contoh: daftar menu dengan filter opsional

```
public function index(Request $request)
{
    $menus = Menu::query()
        ->when($request->filled('q'), fn ($query) => $query->where(
            'name', 'like', '%'.$request->string('q').'%'
        ))
        ->when($request->filled('available'), fn ($query) => $query->where(
            'is_available', $request->boolean('available')
        ))
        ->orderBy('name')
        ->paginate(15)
        ->withQueryString();

    return view('tenant.menus.index', compact('menus'));
}
```

Metode `when()` menambahkan kondisi hanya apabila parameter terkait tersedia, sehingga Anda tidak perlu menyusun rantai `if` yang panjang. Metode `paginate(15)` membatasi jumlah baris pada satu halaman dan menjaga penggunaan memori ketika data bertambah, sedangkan `withQueryString()` mempertahankan parameter pencarian pada tautan halaman berikutnya.

5.5.3 Update: Memperbarui Data dengan Policy

Pembaruan data menggunakan model yang telah dibatasi tenant dan *policy* yang memeriksa izin pengguna.

Kode 5.17 — Contoh: pembaruan menu

```php
public function update(UpdateMenuRequest $request, Tenant $tenant, Menu $menu)
{
    $this->authorize('update', $menu);

    $menu->update($request->validated());

    return redirect()
        ->route('tenant.menus.index', $tenant)
        ->with('ok', 'Menu diperbarui.');
}
```

Dua mekanisme bekerja berurutan di sini. Karena `Menu` memakai *global scope* tenant, model milik tenant lain tidak ditemukan dalam konteks tenant aktif sehingga Laravel mengembalikan 404. Setelah model ditemukan, `authorize()` memastikan pengguna memang berhak memperbaruinya.

5.5.4 Delete dan Soft Delete

Penghapusan fisik tidak selalu tepat, terutama jika data masih diperlukan untuk riwayat transaksi, audit, atau pemulihan kesalahan. *Soft delete* menandai data sebagai terhapus melalui kolom `deleted_at`. Data tersebut tidak muncul pada kueri biasa, tetapi masih dapat dipulihkan apabila kebijakan aplikasi mengizinkannya.

Kode 5.18 — Soft delete pada model dan migration

```php
// Model
class Tenant extends Model
{
    use SoftDeletes;
}

// Migration
$table->softDeletes('deleted_at', 6);
```

Pada proyek pendamping, `softDeletes()` dipasang pada tabel `tenants`. Pilihannya masuk akal: menghapus tenant secara permanen akan memutus rujukan dari menu, pesanan, dan pembayaran yang masih diperlukan untuk rekonsiliasi keuangan.

Dalam operasi sehari-hari, pengelola kantin bahkan tidak perlu menghapus tenant. Proyek pendamping menyediakan aksi **nonaktifkan** yang hanya mengubah kolom `status` menjadi `inactive`: tenant hilang dari katalog pelanggan, tetapi seluruh data historisnya tetap utuh. Implementasinya dibahas pada Subbab 5.6.2 sebagai contoh *service*.

> **Catatan.** *Soft delete* bukan pengganti kebijakan retensi data. Data yang ditandai terhapus tetap tersimpan. Aplikasi perlu menentukan kapan data dipulihkan, diarsipkan, atau dihapus permanen sesuai kebutuhan bisnis dan ketentuan yang berlaku.

5.6 Controller Ramping: Memindahkan Logika ke Service

Sejauh ini, setiap contoh *controller* pada bab ini relatif pendek. Namun, seiring bertambahnya fitur, *controller* cenderung menampung semakin banyak pekerjaan: memvalidasi masukan, menyimpan data, mengirim surel, mencatat audit, dan seterusnya. Subbab ini membahas cara memindahkan

logika tersebut ke kelas yang lebih tepat dengan mengacu pada artikel Korop (2022) di Laravel News, lalu membandingkannya dengan cara proyek pendamping menata modul Admin.

5.6.1 Gejala Controller Gemuk

Korop (2022) memulai pembahasannya dengan *controller* registrasi yang mengerjakan tujuh hal dalam satu metode. Kode 5.19 menyajikan versi ringkasnya.

Kode 5.19 — Contoh generik: controller registrasi yang gemuk (diadaptasi dari Korop, 2022)

```php
public function store(Request $request)
{
    // 1. Validasi
    $request->validate([
        'name' => ['required', 'string', 'max:255'],
        'email' => ['required', 'string', 'email', 'max:255', 'unique:users'],
        'password' => ['required', 'confirmed', Password::defaults()],
    ]);

    // 2. Unggah avatar
    $avatar = $request->hasFile('avatar')
        ? $request->file('avatar')->store('avatars')
        : null;

    // 3. Membuat pengguna
    $user = User::create([
        'name' => $request->name,
        'email' => $request->email,
        'password' => Hash::make($request->password),
        'avatar' => $avatar,
    ]);

    // 4. Login
    Auth::login($user);

    // 5. Membuat voucher dan 6. mengirim surel sambutan
    $voucher = Voucher::create([
        'code' => Str::random(8), 'discount_percent' => 10, 'user_id' => $user->id,
    ]);
    $user->notify(new NewUserWelcomeNotification($voucher->code));

    // 7. Memberi tahu seluruh admin
    foreach (config('app.admin_emails') as $adminEmail) {
        Notification::route('mail', $adminEmail)
            ->notify(new NewUserAdminNotification($user));
    }

    return redirect(RouteServiceProvider::HOME);
}
```

Kode tersebut berfungsi, tetapi mengandung tiga masalah. Pertama, metode `store()` memiliki tujuh alasan untuk berubah — dari aturan kata sandi hingga daftar surel admin — sehingga melanggar *single responsibility principle* (SRP). Kedua, logikanya terkunci pada HTTP: bila pengguna yang sama perlu dibuat dari perintah Artisan, *seeder*, atau API, kodenya harus disalin. Ketiga, pengujiannya berat karena setiap skenario harus melewati permintaan HTTP lengkap.

Laravel tidak menetapkan satu tempat baku untuk logika bisnis. Korop menegaskan bahwa struktur proyek adalah keputusan tim, lalu menawarkan beberapa tempat pemindahan yang masing-masing cocok untuk jenis pekerjaan tertentu. Tabel 5.6 memetakan tempat-tempat tersebut ke padanannya pada proyek pendamping.

Jenis pekerjaan	Tempat menurut Korop (2022)	Padanan pada proyek pendamping
Validasi dan otorisasi masukan	Form Request	`StoreTenantRequest`, `ScheduleCommissionRequest`, `StoreBankAccountRequest`
Operasi bisnis yang terdiri atas beberapa langkah	Service class	`CreateTenant`, `ChangeTenantStatus`, `ChangeCommissionSchedule`, `ManageBankAccount`, `AssignTenantRole`
Satu operasi yang berdiri sendiri	Action class	`CreateNewUser` milik Fortify (Kode 5.1)
Pekerjaan berat di latar belakang	Job pada antrean	`CountTenantMenus` (contoh job sadar tenant)
Reaksi pasif setelah sebuah peristiwa	Event dan listener	Belum dipakai; pengiriman tautan atur kata sandi ditunda dengan `DB::afterCommit()`
Reaksi otomatis atas perubahan model	Observer	Sengaja tidak dipakai; audit dipanggil eksplisit melalui `AuditLogger`

Tabel 5.6. Tempat memindahkan logika dari controller dan padanannya pada proyek pendamping.

Peringatan. Korop sendiri menyebut *observer* sebagai pola yang agak berbahaya: kodenya berjalan diam-diam setiap kali model disimpan, sehingga pembaca *controller* tidak melihat adanya efek samping. Proyek pendamping memilih memanggil `AuditLogger` secara eksplisit di dalam *service* agar setiap pencatatan audit terlihat ketika kode ditelusuri.

5.6.2 Pembagian Tugas Controller dan Service

Pada proyek pendamping, setiap modul menyimpan logika aplikasinya di folder `app/Modules/{Modul}/Services`. Dokumen arsitektur proyek menetapkan arah dependensi: lapisan antarmuka (rute, *controller*, Blade) memanggil lapisan aplikasi (*service*), bukan sebaliknya, dan *service* tidak bergantung pada HTTP. Fitur mengaktifkan dan menonaktifkan tenant memperlihatkan pembagian ini dengan jelas. Berkas *controller*-nya adalah `app/Modules/Admin/Http/Controllers/AdminTenantStatusController.php`.

Kode 5.20 — app/Modules/Admin/Http/Controllers/AdminTenantStatusController.php

```php
class AdminTenantStatusController extends Controller
{
    use AuthorizesRequests;

    public function __construct(private ChangeTenantStatus $service) {}

    public function __invoke(Request $request, Tenant $tenant): RedirectResponse
    {
        $this->authorize('update', $tenant);
        $data = $request->validate([
            'status' => ['required', Rule::in(ChangeTenantStatus::ALLOWED)],
        ]);

        $this->service->handle($tenant, $data['status']);

        return redirect()->route('admin.tenants.index')->with('status', $data['status'] ===
'active'
            ? "Tenant {$tenant->display_name} diaktifkan."
            : "Tenant {$tenant->display_name} dinonaktifkan dan disembunyikan dari katalog.");
```

```
    }
}
```

Controller tersebut hanya mengurus HTTP: memeriksa izin melalui policy, memvalidasi masukan, memanggil *service*, lalu menyusun respons pengalihan beserta pesannya. Ia tidak tahu bagaimana status disimpan atau dicatat. Pekerjaan itu milik *service* `app/Modules/Admin/Services/ChangeTenantStatus.php`.

Kode 5.21 — app/Modules/Admin/Services/ChangeTenantStatus.php

```php
final class ChangeTenantStatus
{
    public const ALLOWED = ['active', 'inactive'];

    public function __construct(private AuditLogger $audit) {}

    public function handle(Tenant $tenant, string $status): Tenant
    {
        if (! in_array($status, self::ALLOWED, true)) {
            throw new DomainException('Status tenant tidak dikenal.');
        }

        return DB::transaction(function () use ($tenant, $status): Tenant {
            $locked = Tenant::query()->lockForUpdate()->findOrFail($tenant->id);
            $before = $locked->status;

            if ($before === $status) {
                return $locked;
            }

            $locked->forceFill(['status' => $status])->save();

            $this->audit->record('tenant', $locked->id,
                $status === 'active' ? 'activated' : 'deactivated',
                ['status' => $before], ['status' => $status],
                $locked->id, $locked->canteen_id);

            return $locked;
        });
    }
}
```

Bandingkan Kode 5.20 dan Kode 5.21 melalui empat pengamatan berikut.

- *Service* menerima model `Tenant` dan string status, bukan objek `Request`. Karena itu, *service* yang sama dapat dipanggil dari *controller*, perintah Artisan, *job*, atau pengujian tanpa perlu memalsukan permintaan HTTP.
- Konstanta `ALLOWED` dipakai dua kali: sebagai aturan validasi pada *controller* dan sebagai penjaga terakhir di dalam *service*. Daftar status yang sah cukup ditulis di satu tempat.
- Batas transaksi, kunci baris melalui `lockForUpdate()`, dan pencatatan audit berada di *service*. *Controller* lain yang kelak memanggil *service* ini otomatis memperoleh perlindungan yang sama tanpa perlu mengingat urutan langkahnya.
- *Service* bersifat idempoten: bila status yang diminta sudah berlaku, ia langsung mengembalikan tenant tanpa menulis audit ganda.

Menonaktifkan tenant juga bukan menghapusnya. Tenant nonaktif hilang dari katalog publik karena `PublicCatalogQuery` hanya membaca tenant berstatus `active`, tetapi menu, pesanan, dan ledger-nya tetap utuh — sejalan dengan pembahasan *soft delete* pada Subbab 5.5.4.

5.6.3 Dependency Injection dan Komposisi Service

Korop (2022) menyuntikkan *service* melalui parameter metode, misalnya `store(StoreUserRequest $request, UserService $userService)`. Proyek pendamping memakai cara lain yang setara, yaitu *constructor injection* dengan *promoted property* PHP 8. *Service container* Laravel membaca tipe parameter konstruktor, membuat objeknya, lalu menyerahkannya ke *controller* secara otomatis.

Kode 5.22 — Constructor injection pada controller dan service (potongan)

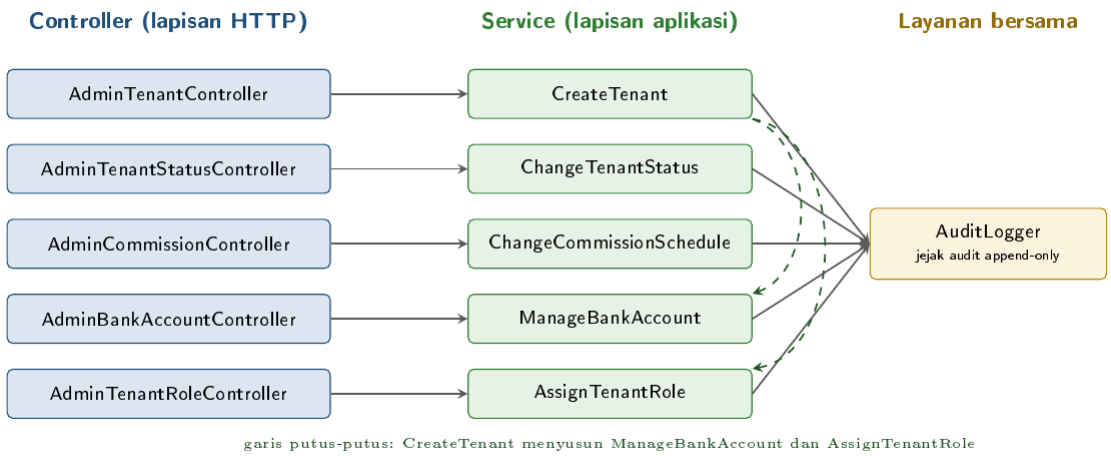
```php
// app/Modules/Admin/Http/Controllers/AdminTenantController.php
final class AdminTenantController extends Controller
{
    public function __construct(private CreateTenant $createTenant) {}

    public function store(StoreTenantRequest $request): RedirectResponse
    {
        $canteen = $this->canteen($request);
        $data = $request->validated();
        $tenant = $this->createTenant->handle($canteen, [
            'code' => $data['code'],
            'slug' => $data['slug'],
            'display_name' => $data['display_name'],
            'commission_rate' => (float) $data['commission_rate'] / 100, // persen -> fraksi
            'pic_name' => $data['pic_name'],
            'pic_email' => $data['pic_email'],
            'bank_code' => $data['bank_code'],
            'account_holder' => $data['account_holder'],
            'account_number' => $data['account_number'],
        ]);

        return redirect()->route('admin.tenants.edit', $tenant)
            ->with('status', "Tenant dibuat. Tautan atur kata sandi dikirim ke
{$data['pic_email']}.");
    }
}

// app/Modules/Admin/Services/CreateTenant.php
final class CreateTenant
{
    public function __construct(
        private AuditLogger $audit,
        private ManageBankAccount $bankAccounts,
        private AssignTenantRole $roles,
    ) {}
}
```

Rantai dependensinya bertingkat. `AdminTenantController` membutuhkan `CreateTenant`; `CreateTenant` membutuhkan `AuditLogger`, `ManageBankAccount`, dan `AssignTenantRole`; dua yang terakhir juga membutuhkan `AuditLogger`. Tidak satu pun kelas tersebut memanggil `new` untuk dependensinya — *container* menyusun seluruh graf itu sendiri. Gambar 5.7 memperlihatkan hubungan tersebut.



Gambar 5.7. Graf dependensi controller dan service pada modul Admin.

Sumber: disusun penulis dari repositori pendamping (tag pertemuan-05).

Komposisi ini mencegah aturan tertulis dua kali. Saat *onboarding*, `CreateTenant` tidak menyimpan rekening bank sendiri, tetapi meminjam `ManageBankAccount::store()` — metode yang sama dengan yang dipakai formulir tambah rekening. Aturan enkripsi nomor rekening dan penyimpanan empat digit terakhirnya cukup dipelihara di satu tempat.

Manfaat lain terasa pada pengujian. Berkas `tests/Feature/CommissionScheduleBoundaryTest.php` memanggil *service* secara langsung melalui `app(ChangeCommissionSchedule::class)->handle(...)`, tanpa login dan tanpa permintaan HTTP, sehingga pengujian dapat berfokus sepenuhnya pada aturan batas waktu.

> **Tip.** Laravel tidak menyediakan perintah khusus untuk membuat *service*. Buat kelasnya dengan `php artisan make:class` atau secara manual. *Container* tetap dapat menyuntikkannya tanpa registrasi apa pun, karena kelas konkret yang seluruh parameter konstruktornya berupa kelas dapat diselesaikan otomatis (*autowiring*).

5.6.4 Service atau Action?

Korop membedakan dua bentuk. *Service class* mengelompokkan beberapa metode yang berkaitan, misalnya `UserService` dengan `uploadAvatar()`, `createUser()`, dan `sendWelcomeEmail()`. *Action class* hanya mengerjakan satu operasi melalui satu metode, misalnya `CreateNewUser::handle()`. Proyek pendamping ternyata memakai keduanya di dalam folder `Services`.

- Bergaya *action*: `CreateTenant`, `ChangeTenantStatus`, dan `ChangeCommissionSchedule` — namanya berupa kata kerja dan hanya memiliki satu metode publik, `handle()`.
- Bergaya *service*: `ManageBankAccount` dengan `store()`, `verify()`, dan `makePrimary()`, serta `AssignTenantRole` dengan `assign()` dan `remove()` — beberapa metode yang berbagi entitas dan dependensi yang sama.

Keduanya sah; yang menentukan adalah kesepakatan tim dan konsistensi. Sebagai pegangan, gunakan bentuk *action* ketika operasinya terdiri atas banyak langkah dan layak diberi nama sendiri, dan bentuk *service* ketika beberapa operasi kecil berbagi aturan serta dependensi yang sama. Apa pun bentuknya, prinsip berikut tetap berlaku.

- *Service* tidak bergantung pada `Request`. Contoh `uploadAvatar(Request $request)` pada artikel Korop masih menerima objek HTTP; pada proyek pendamping, *controller* menyerahkan data yang sudah divalidasi (`$request->validated()`) atau model hasil *route model binding*.
- *Service* menyatakan pelanggaran aturan bisnis dengan *exception* domain, sedangkan *controller* menerjemahkannya menjadi respons HTTP (Kode 5.23).
- Batas transaksi (`DB::transaction()`) ditetapkan di *service*, karena hanya *service* yang tahu langkah mana yang harus berhasil atau gagal bersama-sama.
- Efek samping ke luar sistem, seperti mengirim surel, dijalankan setelah transaksi berhasil melalui `DB::afterCommit()` — sejalan dengan saran Korop untuk memisahkan operasi aktif dari operasi pasif.

Kode 5.23 — app/Modules/Admin/Http/Controllers/AdminTenantRoleController.php (potongan)

```php
public function destroy(Request $request, Tenant $tenant, User $user): RedirectResponse
{
    $this->authorize('assignRole', $tenant);
    $data = $request->validate(['role' => ['required', 'in:owner,operator,cashier']]);

    try {
        $this->service->remove($tenant, $user, $data['role']);
    } catch (DomainException $e) {
        return redirect()->route('admin.tenants.edit', $tenant)
            ->withErrors(['role' => $e->getMessage()]);
    }

    return redirect()->route('admin.tenants.edit', $tenant)->with('status', 'Role dicabut.');
}
```

Metode `AssignTenantRole::remove()` (Kode 5.25) tidak tahu apa pun tentang sesi atau pengalihan; ia hanya melempar `DomainException` ketika owner terakhir hendak dicabut. *Controller*-lah yang memutuskan bahwa kegagalan itu ditampilkan sebagai pesan galat pada formulir. Bila kelak *service* yang sama dipanggil dari API, *controller* API dapat menerjemahkannya menjadi respons JSON berstatus 422 tanpa mengubah *service* sedikit pun.

> **Catatan.** Prinsip di atas bukan aturan mutlak. `AuditLogger` masih membaca *header* `X-Request-Id` melalui helper `request()`. Ini kompromi pragmatis: pengenal permintaan memang hanya ada dalam konteks HTTP, dan *service* tetap dapat berjalan di luar HTTP karena memakai UUID baru sebagai cadangan.

> **Peringatan.** Memindahkan kode ke *service* bukan tujuan akhir. *Service* yang menampung puluhan metode tak berkaitan hanya memindahkan *controller* gemuk ke tempat lain. Terapkan SRP pada *service* seperti pada *controller*: satu kelas, satu alasan untuk berubah.

5.7 Administrasi Entitas dan Penugasan Peran

Modul administrasi mengelola entitas seperti tenant, staf, tarif komisi, dan rekening bank. Karena perubahan pada bagian ini dapat memengaruhi hak akses dan keuangan banyak pihak, modul administrasi membutuhkan kontrol yang lebih ketat daripada CRUD biasa.

5.7.1 Membuat Tenant Secara Atomik

Membuat tenant bukan sekadar menyisipkan satu baris. Sesuai alur *onboarding* pada proyek pendamping, tenant baru harus langsung memiliki skema komisi aktif dan saldo awal — jika tidak,

transaksi pertamanya gagal dihitung — serta akun penanggung jawab (owner) dan rekening bank utama. Berkasnya adalah `app/Modules/Admin/Services/CreateTenant.php`.

Kode 5.24 — app/Modules/Admin/Services/CreateTenant.php (metode handle)

```
public function handle(Canteen $canteen, array $data): Tenant
{
    return DB::transaction(function () use ($canteen, $data): Tenant {
        $tenant = (new Tenant)->forceFill([
            'canteen_id' => $canteen->id,
            'code' => $data['code'],
            'slug' => $data['slug'],
            'display_name' => $data['display_name'],
            'status' => 'active',
        ]);
        $tenant->save();

        (new TenantBalance)->forceFill([
            'tenant_id' => $tenant->id,
            'available_amount' => 0,
            'held_amount' => 0,
        ])->save();

        $commission = (new CommissionScheme)->forceFill([
            'tenant_id' => $tenant->id,
            'commission_rate' => $data['commission_rate'],
            'valid_from' => now(),
            'valid_to' => null,
        ]);
        $commission->save();

        $this->audit->record('tenant', $tenant->id, 'created',
            null, $tenant->only(['code', 'slug', 'display_name', 'status']),
            $tenant->id, $canteen->id);
        $this->audit->record('commission_scheme', $commission->id, 'created',
            null, $commission->only(['commission_rate', 'valid_from']), $tenant->id, $canteen-
>id);

        if (isset($data['pic_email'])) {
            $owner = (new User)->forceFill([
                'name' => $data['pic_name'],
                'email' => $data['pic_email'],
                'password' => Str::password(32),
                'role' => 'tenant',
                'status' => 'active',
                'email_verified_at' => now(), // diverifikasi pengelola saat onboarding
            ]);
            $owner->save();
            $this->roles->assign($tenant, $owner, 'owner');

            DB::afterCommit(fn () => Password::broker()->sendResetLink(['email' => $owner-
>email]));
        }

        if (isset($data['account_number'])) {
            $this->bankAccounts->store($tenant, $data)->forceFill(['is_primary' => true])-
>save();
        }

        return $tenant;
    });
}
```

Seluruh langkah dibungkus `DB::transaction()`. Jika salah satu gagal, tidak ada satu pun yang tersimpan — sehingga tidak mungkin ada tenant tanpa skema komisi atau tanpa owner. Perhatikan

juga penggunaan `forceFill()`: kolom seperti `tenant_id` dan `status` sengaja tidak *mass-assignable*, sehingga hanya lapisan *service* yang dipercaya boleh menetapkannya.

Dua keputusan lain layak dicermati. Pertama, akun owner dibuat dengan kata sandi acak `Str::password(32)` yang tidak pernah diketahui siapa pun; pemilik akun menerima tautan atur kata sandi melalui *password broker*, bukan kata sandi mentah lewat surel. Kedua, tautan itu dikirim di dalam `DB::afterCommit()`, sehingga surel hanya terkirim bila transaksi benar-benar tersimpan. Tanpa penundaan ini, kegagalan pada langkah terakhir akan membatalkan data, tetapi surelnya telanjur terkirim untuk akun yang tidak pernah ada. Penyimpanan rekening dan penugasan owner sendiri didelegasikan ke `ManageBankAccount` dan `AssignTenantRole`, seperti dibahas pada Subbab 5.6.3.

5.7.2 Penugasan dan Pencabutan Peran

Relasi pengguna dan peran tenant berbentuk banyak-ke-banyak melalui tabel `user_tenant_roles`. Berkas layanannya adalah `app/Modules/Admin/Services/AssignTenantRole.php`.

Kode 5.25 — app/Modules/Admin/Services/AssignTenantRole.php

```php
public function assign(Tenant $tenant, User $user, string $role): UserTenantRole
{
    $membership = UserTenantRole::firstOrCreate([
        'user_id' => $user->id,
        'tenant_id' => $tenant->id,
        'role' => $role,
    ]);

    $this->audit->record('user_tenant_role', $membership->id, 'assigned',
        null, ['user_id' => $user->id, 'role' => $role],
        $tenant->id, $tenant->canteen_id);

    return $membership;
}

public function remove(Tenant $tenant, User $user, string $role): void
{
    if ($role === 'owner') {
        $owners = UserTenantRole::query()
            ->where('tenant_id', $tenant->id)->where('role', 'owner')->count();
        if ($owners <= 1) {
            throw new DomainException('Tidak boleh menghapus owner terakhir tenant.');
        }
    }

    UserTenantRole::query()
        ->where('user_id', $user->id)
        ->where('tenant_id', $tenant->id)
        ->where('role', $role)
        ->delete();

    $this->audit->record('user_tenant_role', null, 'removed',
        ['user_id' => $user->id, 'role' => $role], null,
        $tenant->id, $tenant->canteen_id);
}
```

Perhatikan tiga keputusan desain pada Kode 5.25. Pertama, `firstOrCreate()` membuat penugasan bersifat idempoten: menugaskan peran yang sama dua kali tidak menghasilkan baris ganda. Kedua, pencabutan peran `owner` dijaga agar tenant tidak pernah kehilangan pemilik terakhirnya — tanpa penjagaan itu, sebuah tenant dapat menjadi yatim dan tidak lagi dapat dikelola siapa pun. Ketiga, kedua tindakan dicatat ke *audit trail*.

Tip. Bandingkan `firstOrCreate()` dengan `sync()`. Gunakan `sync()` ketika formulir memang dimaksudkan menetapkan ulang **seluruh** peran pengguna sekaligus, karena peran yang tidak ada pada daftar akan dilepas. Gunakan `firstOrCreate()`, `attach()`, atau `detach()` ketika hanya ingin menambah atau mencabut satu peran tertentu.

5.7.3 Menghindari Konflik Peran

Tidak semua kombinasi peran aman. Sistem dapat melarang satu pengguna memiliki peran yang mengubah tarif komisi sekaligus menyetujui perubahan tarif tersebut. Sebelum menyimpan penugasan peran, aplikasi perlu memeriksa:

- apakah administrator berwenang mengubah peran pengguna tersebut;
- apakah pengguna dan peran berada dalam tenant atau kantin yang sesuai;
- apakah kombinasi peran melanggar pemisahan tugas;
- apakah perubahan peran tercatat dalam *audit trail*.

Tip. Tetapkan daftar kombinasi peran yang dilarang berdasarkan proses bisnis nyata. Jangan membuat larangan secara acak; dasarkan aturan pada risiko kecurangan, konflik kepentingan, dan kebutuhan operasional.

5.8 Aturan Berlaku Efektif dan Audit Trail

Beberapa aturan bisnis berubah menurut waktu. Tarif komisi, pajak layanan, atau kebijakan diskon dapat memiliki nilai berbeda pada periode berbeda. Menimpa nilai lama akan menghilangkan dasar perhitungan historis, sehingga sistem perlu menyimpan versi aturan beserta masa berlakunya.

5.8.1 Konvensi Rentang Berlaku

Proyek pendamping menggunakan konvensi berikut pada tabel `commission_schemes`:

- `valid_from` bersifat inklusif: aturan mulai berlaku tepat pada saat itu;
- `valid_to` bersifat eksklusif: saat itu adalah instan pertama aturan *tidak* berlaku lagi, sekaligus `valid_from` versi penggantinya;
- `valid_to` bernilai null berarti aturan masih berlaku hingga sekarang;
- keduanya bertipe timestamp dengan presisi mikrodetik, bukan tanggal.

Dengan kata lain, setiap versi berlaku pada interval setengah-terbuka [`valid_from`, `valid_to`). Predikat aturan aktif pada waktu tertentu dapat dinyatakan sebagai berikut.

$$\text{berlaku}(a, t) \iff \mathit{valid_from}(a) \le t \ \wedge\ \big(\mathit{valid_to}(a) = \varnothing \ \vee\ t < \mathit{valid_to}(a)\big)$$

(Persamaan 5.2)

dengan *a* adalah satu versi aturan dan *t* adalah waktu yang ditanyakan. Simbol himpunan kosong menandakan `valid_to` bernilai `null`.

Predikat tersebut diterjemahkan langsung menjadi *local scope* pada model `app/Models/CommissionScheme.php`.

Kode 5.26 — app/Models/CommissionScheme.php (scope effectiveAt)

```php
/**
 * Skema yang berlaku pada instan tertentu, interval setengah-terbuka [valid_from, valid_to).
 */
#[Scope]
protected function effectiveAt(Builder $query, CarbonInterface $instant): void
```

```
{
    $query->where('valid_from', '<=', $instant)
        ->where(function (Builder $query) use ($instant): void {
            $query->whereNull('valid_to')->orWhere('valid_to', '>', $instant);
        });
}
```

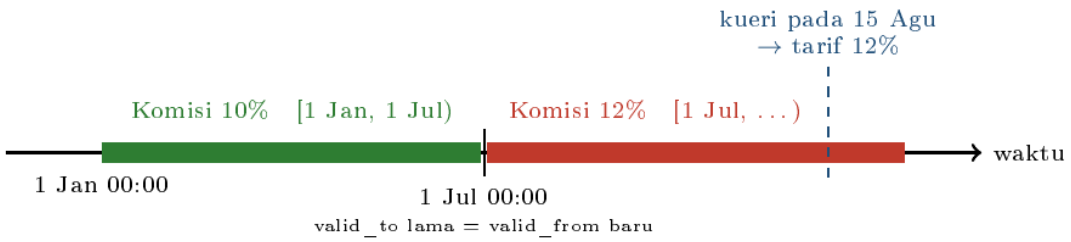
Atribut `#[Scope]` menandai metode tersebut sebagai *local scope*, sehingga tarif yang berlaku cukup ditanyakan melalui `CommissionScheme::query()->effectiveAt(now())->first()`. Halaman daftar tenant memakai scope ini untuk menampilkan komisi yang aktif saat ini.

Ketika tarif baru dijadwalkan, `valid_to` versi lama diisi tepat dengan waktu efektif versi baru. Karena batas akhirnya eksklusif, setiap instan jatuh pada tepat satu versi — tanpa celah maupun tumpang tindih — berapa pun presisi timestamp yang dipakai.

> **Catatan.** Versi awal repositori memakai batas akhir inklusif dan menutup versi lama satu detik sebelum versi baru (`subSecond()`). Karena kolomnya berpresisi mikrodetik, konvensi itu meninggalkan celah: pesanan pada pukul 23:59:59,5 tidak menemukan tarif mana pun. Interval setengah-terbuka menghapus celah tersebut tanpa bergantung pada presisi, dan pada tag pertemuan-05 perilakunya dikunci oleh berkas uji `CommissionScheduleBoundaryTest`.

Versi	Berlaku mulai	Berlaku hingga	Tarif komisi
Lama	1 Januari 2026, 00:00:00	1 Juli 2026, 00:00:00 (tidak termasuk)	0,1000 atau 10%
Baru	1 Juli 2026, 00:00:00	(kosong — masih berlaku)	0,1200 atau 12%

Tabel 5.7. Contoh dua versi tarif komisi dengan batas akhir eksklusif.



Gambar 5.8. Garis waktu dua versi tarif komisi dan tarif yang berlaku pada tanggal kueri.

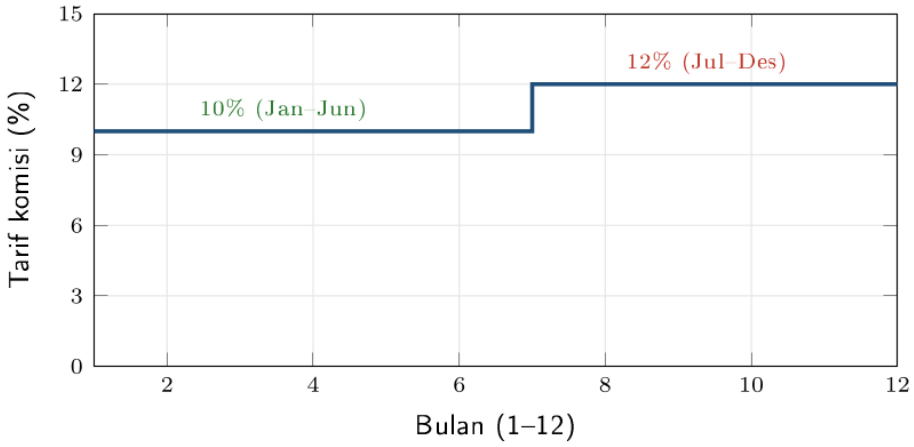
Sumber: diadaptasi penulis, ilustratif.

Komisi sebuah pesanan dihitung dari nilai pesanan dikalikan tarif yang berlaku pada waktu transaksi.

$$c = p \cdot r(t) \tag{Persamaan 5.3}$$

dengan c adalah komisi, p adalah nilai pesanan, dan $r(t)$ adalah tarif yang berlaku pada waktu t.

Contoh 5.2. Untuk pesanan senilai Rp20.000: pada 15 Maret 2026 tarif efektifnya 10%, sehingga komisi adalah Rp2.000. Pada 15 Agustus 2026 tarif efektifnya 12%, sehingga komisi adalah Rp2.400.



Gambar 5.9. Tarif komisi sebagai fungsi tangga terhadap waktu.

Sumber: diadaptasi penulis, data ilustratif.

Catatan. Tarif disimpan sebagai `decimal(6,4)` — misalnya `0.1500` untuk 15% — bukan sebagai bilangan pecahan biner (*float*). Alasannya sama dengan penyimpanan uang pada Bab 3: tipe desimal eksak menghindari galat pembulatan yang menumpuk pada perhitungan keuangan. Formulir administrasi menerima angka persen, lalu *controller* membaginya dengan 100 sebelum disimpan.

5.8.2 Implementasi Penjadwalan Tarif

Berikut struktur tabel yang menyimpan versi tarif. Berkasnya adalah migration `create_catalog_tables`.

Kode 5.27 — Migration commission_schemes (potongan)

```php
Schema::create('commission_schemes', function (Blueprint $table): void {
    $table->id();
    $table->foreignId('tenant_id')->constrained()->cascadeOnDelete();
    $table->decimal('commission_rate', 6, 4); // exact, mis. 0.1500 = 15%
    $table->timestampTz('valid_from', 6);
    $table->timestampTz('valid_to', 6)->nullable(); // null = masih berlaku
    $table->timestamps(6);

    $table->unique(['tenant_id', 'id']); // target FK snapshot komisi
    $table->index(['tenant_id', 'valid_from']);
});
```

Menjadwalkan tarif baru berarti melakukan tiga hal sekaligus: mengunci versi aktif, menutup rentangnya, dan menyisipkan versi baru. Ketiganya harus atomik. Berkasnya adalah `app/Modules/Admin/Services/ChangeCommissionSchedule.php`.

Kode 5.28 — app/Modules/Admin/Services/ChangeCommissionSchedule.php

```php
public function handle(
    Tenant $tenant, int|float|string $rate, CarbonInterface $effectiveAt
): CommissionScheme {
    return DB::transaction(function () use ($tenant, $rate, $effectiveAt): CommissionScheme {
        $active = CommissionScheme::query()
            ->withoutGlobalScope('tenant')
            ->where('tenant_id', $tenant->id)
            ->whereNull('valid_to')
            ->lockForUpdate()
            ->first();
```

```
        if ($active !== null && $active->valid_from >= $effectiveAt) {
            throw new DomainException(
                'Waktu efektif harus setelah versi aktif (mencegah overlap).'
            );
        }

        $before = $active?->only(['id', 'commission_rate', 'valid_from', 'valid_to']);
        // Interval setengah-terbuka [valid_from, valid_to): versi lama berakhir TEPAT
        // saat versi baru mulai berlaku.
        $active?->forceFill(['valid_to' => $effectiveAt])->save();

        $new = (new CommissionScheme)->forceFill([
            'tenant_id' => $tenant->id,
            'commission_rate' => $rate,
            'valid_from' => $effectiveAt,
            'valid_to' => null,
        ]);
        $new->save();

        $this->audit->record('commission_scheme', $new->id, 'scheduled',
            $before, $new->only(['commission_rate', 'valid_from']),
            $tenant->id, $tenant->canteen_id);

        return $new;
    });
}
```

Baca kode tersebut sebagai empat keputusan berlapis.

- `lockForUpdate()` mengunci baris versi aktif sehingga dua permintaan yang bersamaan tidak dapat sama-sama menutup versi yang sama.
- Pemeriksaan `valid_from >= $effectiveAt` menolak penjadwalan mundur yang akan menghasilkan rentang tumpang tindih.
- `valid_to` versi lama diisi tepat `$effectiveAt`, sesuai konvensi batas akhir eksklusif, sehingga versi baru bersambung tanpa celah.
- Seluruh langkah berada dalam satu transaksi, sehingga kegagalan di tengah tidak meninggalkan tenant tanpa tarif aktif.

Lapis terakhir berada di basis data. Migration `add_admin_guards_to_schema` menambahkan kolom terhitung dan indeks unik yang membuat dua versi aktif menjadi mustahil.

Kode 5.29 — Migration add_admin_guards_to_schema (potongan)

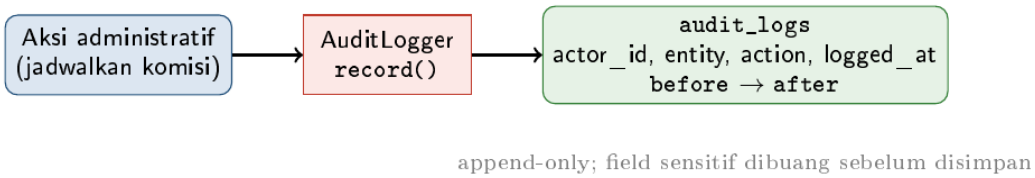
```
Schema::table('commission_schemes', function (Blueprint $table): void {
    $table->unsignedBigInteger('active_lock')
        ->virtualAs('case when `valid_to` is null then `tenant_id` else null end')
        ->nullable();
    $table->unique('active_lock', 'uq_commission_active_per_tenant');
});
```

Kolom `active_lock` bernilai `tenant_id` hanya ketika `valid_to` masih `null`, dan bernilai `null` pada baris lainnya. Karena indeks unik pada MariaDB mengabaikan nilai `null`, aturan “satu versi aktif per tenant” ditegakkan langsung oleh basis data — bahkan terhadap penyisipan yang melewati lapisan aplikasi.

> **Peringatan.** Validasi aplikasi saja tidak mencegah dua tarif aktif pada periode yang sama, terutama pada permintaan yang berjalan bersamaan. Diperlukan kombinasi tiga hal: kunci baris di dalam transaksi, validasi bisnis, dan batasan basis data. Pengujian batas waktu melengkapi ketiganya.

5.8.3 Audit Trail

Audit trail adalah catatan perubahan yang membantu menjawab empat pertanyaan: siapa yang melakukan perubahan, data apa yang berubah, kapan perubahan terjadi, dan dari nilai apa menjadi nilai apa.



Gambar 5.10. Alur pencatatan audit dari aksi administratif hingga tabel audit_logs.

Sumber: diadaptasi penulis, ilustratif.

Kolom	Makna
actor_id	Pengguna pelaku tindakan
canteen_id dan tenant_id	Konteks tempat tindakan terjadi
entity dan entity_id	Jenis dan pengenal data yang dicatat
action	Jenis tindakan, misalnya `created`, `scheduled`, `assigned`
request_id	Pengenal permintaan, untuk menautkan beberapa catatan dari satu aksi
before dan after	Nilai sebelum dan sesudah perubahan, disimpan sebagai JSON
logged_at	Waktu pencatatan

Tabel 5.8. Struktur tabel audit_logs pada proyek pendamping.

Pencatatan dilakukan oleh satu layanan terpusat, bukan disebar di banyak *controller*. Berkasnya adalah `app/Modules/Admin/Services/AuditLogger.php`.

Kode 5.30 — app/Modules/Admin/Services/AuditLogger.php

```php
public function record(
    string $entity, ?int $entityId, string $action,
    ?array $before = null, ?array $after = null,
    ?int $tenantId = null, ?int $canteenId = null
): AuditLog {
    return AuditLog::create([
        'actor_id' => Auth::id(),
        'canteen_id' => $canteenId,
        'tenant_id' => $tenantId,
        'entity' => $entity,
        'entity_id' => $entityId !== null ? (string) $entityId : null,
        'action' => $action,
        'request_id' => request()->header('X-Request-Id') ?? (string) Str::uuid(),
        'before' => $before !== null ? $this->sanitize($before) : null,
        'after' => $after !== null ? $this->sanitize($after) : null,
        'logged_at' => now(),
    ]);
}

private function sanitize(array $data): array
{
    foreach ([
        'account_number_cipher', 'account_number', 'password', 'password_hash',
    ] as $key) {
        unset($data[$key]);
    }
```

```
    return $data;
}
```

Metode `sanitize()` membuang atribut sensitif sebelum data disimpan. Kata sandi, nomor rekening, token, dan kredensial lain tidak boleh masuk ke tabel audit. Tabel audit sering kali memiliki hak baca yang lebih longgar daripada tabel aslinya justru karena tujuannya adalah pemeriksaan — sehingga menyalin rahasia ke dalamnya membatalkan seluruh perlindungan di tempat lain.

Peringatan. *Audit trail* pada tabel biasa tidak otomatis kebal manipulasi. Proyek pendamping memperlakukan `audit_logs` sebagai *append-only*: catatan tidak pernah diperbarui, dan koreksi dilakukan dengan menulis peristiwa baru. Agar lebih tahan manipulasi, aplikasi masih perlu membatasi hak akses basis data, memisahkan akun administratif, mencadangkan data, dan menetapkan kebijakan retensi.

5.9 Studi Kasus Hipotetis dan Pengujian

Sebuah tenant bernama *Kantin Sejahtera* memiliki seorang operator tenant, dua kasir, dan dua staf dapur. Operator ingin menambahkan menu baru, kasir ingin membuat pesanan, dan staf dapur memperbarui status pesanan. Sementara itu, pengelola kantin menjadwalkan perubahan tarif komisi mulai bulan depan.

Rancangan yang tepat mengikuti langkah berikut.

1. Staf mendaftarkan akun dan memverifikasi emailnya.
2. Pengguna masuk melalui autentikasi berbasis sesi, dengan pembatasan laju percobaan.
3. *Middleware* peran memastikan pengguna berada pada konteks rute yang tepat.
4. *Middleware* tenant menentukan tenant aktif dan memverifikasi keanggotaan.
5. *Form Request* memvalidasi data yang dikirim, termasuk relasi yang harus sesuai tenant.
6. *Global scope* membatasi data pada tenant aktif.
7. *Policy* memeriksa tindakan terhadap objek tertentu.
8. *Controller* mendelegasikan operasi bisnis ke *service* yang membungkus transaksi dan audit.
9. Perubahan tenant, peran, dan tarif komisi dicatat dalam *audit trail*.
10. Pengujian memastikan pengguna tidak dapat melewati kontrol melalui URL, formulir, API, atau manipulasi parameter.

Proyek pendamping menyediakan beberapa berkas uji yang meliput skenario tersebut, seperti dirangkum pada Tabel 5.9.

Skenario	Hasil yang diharapkan	Berkas uji
Pengguna tanpa login membuka dasbor	Dialihkan ke halaman login	`DashboardTest`
Pengguna membuka tautan verifikasi email	Tautan sah mengisi `email_verified_at`; *hash* yang salah ditolak	`EmailVerificationTest`
Formulir tanpa token CSRF dikirim	Permintaan ditolak dengan respons 419	Diperiksa manual

Surel tak terdaftar, kata sandi salah, atau akun nonaktif	Pesan galat generik yang sama	`LoginLockoutTest`
Lima kali gagal masuk dalam sepuluh menit	Akun terkunci 15 menit, meski kata sandi berikutnya benar	`LoginLockoutTest`
Pengguna bukan pengelola kantin membuka daftar tenant	Ditolak oleh policy	`AdminManagementTest`
Tenant didaftarkan	Skema komisi aktif, saldo nol, akun owner, rekening utama terenkripsi, dan tautan atur kata sandi ikut terbentuk	`AdminManagementTest`, `TenantOnboardingTest`
Tenant dinonaktifkan	Hilang dari katalog publik; data historis dan catatan audit tetap ada	`TenantOnboardingTest`
Dua skema komisi aktif disisipkan langsung	Ditolak oleh batasan unik basis data	`AdminManagementTest`
Tarif baru dijadwalkan	Versi lama berakhir tepat saat versi baru berlaku	`CommissionScheduleBoundaryTest`
Tarif ditanyakan tepat di sekitar batas pergantian	Tepat satu skema berlaku pada setiap instan	`CommissionScheduleBoundaryTest`
Owner terakhir tenant dicabut perannya	Ditolak oleh aturan bisnis	`AdminManagementTest`
Nomor rekening disimpan	Tersimpan terenkripsi; hanya empat digit terakhir yang terlihat	`AdminManagementTest`
Aksi administratif sensitif dilakukan	Audit mencatat pelaku serta nilai lama dan baru tanpa data sensitif	`AdminManagementTest`

Tabel 5.9. Skenario pengujian minimum untuk identitas, akses, dan administrasi.

Jalankan berkas uji tersebut di terminal dengan perintah berikut.

Kode 5.31 — Menjalankan pengujian autentikasi dan administrasi (di terminal)

```
php artisan test --
filter="AdminManagementTest|TenantOnboardingTest|CommissionScheduleBoundaryTest|LoginLockoutTe
st"
```

Skenario CSRF pada Tabel 5.9 tidak memiliki uji otomatis karena Laravel sengaja melewati pemeriksaan token CSRF ketika aplikasi berjalan dalam mode pengujian. Periksa skenario itu secara manual, misalnya dengan menghapus direktif `@csrf` dari formulir lalu mengirimkannya.

Rangkuman Bab

- Registrasi membuat akun, verifikasi email memeriksa kepemilikan alamat, autentikasi membuktikan identitas, dan otorisasi menentukan tindakan yang boleh dilakukan. Urutannya tidak dapat dibalik.
- Kata sandi disimpan sebagai *hash* satu arah; aturan kekuatannya dipusatkan agar dapat diperketat sekaligus.
- Pembatasan laju per alamat IP dan penguncian per akun melengkapi kata sandi yang kuat, karena kata sandi terbaik pun dapat ditebak bila percobaannya tak terbatas.

- Proteksi CSRF penting untuk formulir web berbasis sesi karena peramban menyertakan *cookie* sesi secara otomatis.
- RBAC memetakan pengguna ke peran dan izin, mengikuti prinsip *least privilege* dan pemisahan tugas — bukan alasan memperbanyak peran.
- *Gate* cocok untuk kemampuan umum; *policy* cocok untuk tindakan terhadap objek tertentu karena dapat memeriksa atribut objek.
- Operasi CRUD harus menggabungkan autentikasi, otorisasi, validasi, dan isolasi tenant sekaligus.
- *Controller* sebaiknya ramping: validasi di Form Request, logika bisnis di *service* atau *action* yang tidak bergantung pada HTTP, dan dependensinya disuntikkan oleh *service container*.
- Relasi lintas tenant, seperti kategori menu, harus divalidasi berdasarkan tenant aktif — bukan hanya berdasarkan keberadaan ID.
- *Soft delete* membantu pemulihan dan histori, tetapi tidak menggantikan kebijakan retensi data.
- Aturan berlaku efektif menjaga nilai historis. Interval setengah-terbuka [`valid_from`, `valid_to`) menjamin tepat satu versi berlaku pada setiap instan, dan sistem harus mencegah rentang yang tumpang tindih melalui kunci baris, validasi bisnis, dan batasan basis data.
- *Audit trail* membantu akuntabilitas, tetapi harus membuang data sensitif dan diperlakukan sebagai catatan *append-only*.

Bab berikutnya melanjutkan ke sisi pelanggan: identifikasi meja melalui kode QR, token opaque, dan sesi pelanggan anonim — konteks yang justru tidak memiliki akun pengguna, sehingga menuntut mekanisme identitas yang berbeda dari bab ini.

Istilah Penting

Istilah	Makna ringkas
Registrasi	Proses pembuatan akun pengguna baru
Verifikasi email	Pembuktian bahwa pengguna menguasai alamat email
Autentikasi	Pembuktian identitas pengguna
Otorisasi	Penentuan tindakan yang boleh dilakukan pengguna
Hash	Fungsi satu arah untuk menyimpan kata sandi
Session fixation	Serangan dengan memaksa ID sesi yang dipakai korban
Rate limiting	Pembatasan jumlah percobaan dalam satu rentang waktu
Penguncian akun	Pemblokiran sementara sebuah akun setelah sejumlah percobaan masuk gagal
CSRF	Serangan yang memanfaatkan sesi pengguna untuk permintaan palsu
RBAC	Kontrol akses berbasis peran
Least privilege	Pemberian izin seminimal yang diperlukan
Separation of duty	Pemisahan tugas untuk mencegah konflik kepentingan
Gate	Mekanisme otorisasi untuk kemampuan umum

Policy	Mekanisme otorisasi untuk tindakan atas objek tertentu
Form Request	Kelas yang memuat aturan validasi dan otorisasi permintaan
Controller gemuk	Controller yang menampung logika bisnis sehingga sulit dibaca, dipakai ulang, dan diuji
Service class	Kelas lapisan aplikasi yang menampung logika bisnis tanpa bergantung pada HTTP
Action class	Kelas yang mengerjakan tepat satu operasi melalui satu metode publik
Dependency injection	Penyerahan dependensi sebuah kelas oleh service container, bukan dibuat sendiri dengan new
Soft delete	Penandaan data terhapus tanpa penghapusan fisik
Effective dating	Penyimpanan versi aturan beserta masa berlakunya
Interval setengah-terbuka	Rentang [awal, akhir) yang memuat titik awal tetapi tidak memuat titik akhir
Audit trail	Catatan perubahan yang menautkan pelaku, waktu, dan nilai

Tabel 5.10. Istilah penting pada Bab 5.

Pertanyaan dan Latihan

A. Pemahaman

1. Jelaskan perbedaan registrasi, verifikasi email, autentikasi, otorisasi, dan validasi.
2. Mengapa kata sandi perlu disimpan sebagai *hash*, bukan sebagai teks polos?
3. Jelaskan mengapa menyembunyikan tombol pada antarmuka tidak cukup untuk menegakkan otorisasi.
4. Apa perbedaan antara *Gate* dan *policy*, dan kapan masing-masing lebih tepat digunakan?
5. Mengapa `hasRole()` pada proyek pendamping memeriksa status akun, bukan hanya kolom `role`?

B. Penerapan

6. Buat rancangan aturan *Form Request* untuk memperbarui menu dengan atribut `name`, `price_amount`, `stock_qty`, dan `category_id`. Pastikan `category_id` hanya dapat merujuk kategori pada tenant aktif.
7. Tulis langkah *logout* aman pada aplikasi web berbasis sesi dan jelaskan fungsi setiap langkahnya.
8. Untuk $U = 200$, $P = 30$, dan $R = 8$, hitung A(langsung) dan A(RBAC) menggunakan Persamaan 5.1, lalu hitung persentase penghematan ilustratifnya.
9. Berdasarkan Tabel 5.7, hitung komisi untuk pesanan senilai Rp45.000 pada 10 Februari 2026 dan pada 20 September 2026.
10. Tambahkan metode `delete()` pada `TenantPolicy` yang hanya mengizinkan peran `owner` kantin, lalu jelaskan mengapa daftar perannya berbeda dari `scheduleCommission()`.
11. Metode `update()` pada `TenantMenuController` (modul Catalog) masih memperbarui menu langsung di dalam *controller*. Refaktor logika tersebut ke *service* `ToggleMenuAvailability` yang juga mencatat perubahan ke audit trail. Tunjukkan kode *controller* sesudah refaktor, lalu jelaskan apa yang tetap tinggal di *controller* dan mengapa.

C. Analisis dan Evaluasi

12. Audit potongan kode berikut. Jelaskan risikonya pada aplikasi multi-tenant dan usulkan perbaikannya: `'category_id' => ['required', 'exists:menu_categories,id']`.
13. Seorang pengguna memiliki peran kasir sekaligus operator tenant. Analisis manfaat dan risikonya berdasarkan prinsip pemisahan tugas.
14. Sebuah tenant memiliki dua skema komisi: 10% berlaku sejak 1 Januari 2026 tanpa batas akhir, dan 12% berlaku sejak 15 Juni 2026 tanpa batas akhir. Jelaskan masalah yang muncul, lalu uraikan tiga mekanisme pada Subbab 5.8 yang seharusnya mencegahnya.
15. Sebuah tim mengusulkan agar `AuditLogger` mencatat seluruh atribut model yang berubah, tanpa penyaringan. Jelaskan data apa yang sebaiknya tidak dicatat dan mengapa.
16. Layanan `ChangeCommissionSchedule` memakai `withoutGlobalScope('tenant')`. Kaitkan dengan pembahasan *bypass* pada Bab 4: mengapa penggunaan itu dapat dibenarkan di sini, dan kontrol apa yang menggantikannya?
17. `ManageBankAccount` memiliki tiga metode publik, sedangkan `CreateTenant` hanya memiliki `handle()`. Dengan merujuk pembedaan *service class* dan *action class* menurut Korop (2022), evaluasi apakah `ManageBankAccount` sebaiknya dipecah menjadi tiga *action*. Uraikan dampaknya terhadap keterbacaan, pengujian, dan jumlah berkas. Pertimbangkan pula apakah modul Catalog boleh memakai `AuditLogger` milik modul Admin, atau kelas itu sebaiknya dipindahkan ke `app/Support`.
18. Rancang matriks peran dan izin untuk aplikasi kantin yang memiliki pengelola kantin, operator tenant, kasir, staf dapur, dan auditor. Jelaskan alasan setiap pembagian izin, termasuk mengapa auditor tidak boleh memiliki izin tulis.

Daftar Pustaka

Buku

Anderson, R. (2020). *Security Engineering: A Guide to Building Dependable Distributed Systems* (3rd ed.). Wiley.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

Snodgrass, R. T. (2000). *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann.

Stauffer, M. (2023). *Laravel: Up & Running* (3rd ed.). O'Reilly Media.

Artikel Jurnal dan Standar

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., & Youman, C. E. (1996). Role-based access control models. *Computer*, 29(2), 38–47. https://doi.org/10.1109/2.485845

Grassi, P. A., Garcia, M. E., & Fenton, J. L. (2017). *Digital Identity Guidelines* (NIST Special Publication 800-63-3). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-63-3

Dokumentasi dan Sumber Daring

Korop, P. (2022, 27 Mei). *Restructuring a Laravel Controller using Services, Events, Jobs, Actions, and more*. Laravel News. Diakses dari https://laravel-news.com/controller-refactor

Laravel LLC. (2026). *Authentication, Authorization, Validation, CSRF Protection — Laravel 13.x Documentation*. Diakses dari https://laravel.com/docs/13.x/authorization

Laravel LLC. (2026). *Laravel Fortify Documentation*. Diakses dari https://laravel.com/docs/13.x/fortify

OWASP Foundation. (2021). *OWASP Top 10:2021 — A01 Broken Access Control; A07 Identification and Authentication Failures*. Diakses dari https://owasp.org/Top10/

[PERLU VERIFIKASI: metadata, URL, tanggal akses, edisi, dan DOI seluruh rujukan daring]

Sumber Proyek (Dokumen Internal dan Repositori)

Suwardiyanto, D. (2026). *Kantin Multi-Tenant — Repositori Kode Pendamping*, tag `pertemuan-05`. https://github.com/devitsuwardiyanto/kantin-multi-tenant

Politeknik Negeri Banyuwangi. *SRS Aplikasi Kantin Multi-Tenant (ISO/IEC/IEEE 29148), versi 2*. Dokumen proyek.

Politeknik Negeri Banyuwangi. *Modul Praktikum Kantin Multi-Tenant Laravel 13 — Modul 5: Administrasi Kantin, Tenant, Role, dan Skema Komisi*. Dokumen praktikum.

[PERLU VERIFIKASI: status, versi, dan izin penggunaan dokumen internal dalam buku ajar]

Catatan: seluruh data numerik, diagram, dan studi kasus pada bab ini bersifat ilustratif untuk keperluan pembelajaran. Potongan kode yang menyebutkan nama berkas telah diselaraskan dengan repositori pendamping pada tag pertemuan-05; contoh yang ditandai "contoh generik" disajikan untuk menjelaskan mekanisme dan tidak berasal dari repositori.